PostgreSQL and SQL: Week 1 Theory

Oct 26, 2026 · @Mohammed

Read this top to bottom once, then come back and start writing queries on the shop database. The six topics run in the order you planned.

1. SQL syntax

SQL is a declarative language. You describe the result you want, and the database decides how to get it. You never write loops or say "look at row 1, then row 2." You say "give me the customers from Germany, newest first," and Postgres picks the method. That is the biggest mental shift if you come from Python, and it is why two queries that look different can be the same, and two that look the same can behave very differently.

The shape of a query

A basic read query has up to six clauses:

```
SELECT   columns or expressions
FROM     table
WHERE    row filter
GROUP BY grouping columns
HAVING   group filter
ORDER BY sort
LIMIT    how many rows
```

You write them in that order, but the database does not run them in that order. This is the single most useful thing to understand about SQL, and most beginner confusion comes from missing it.

The order a query actually runs in

Logically, Postgres works like this:

1. **FROM** (and JOINs): build the set of rows to work with.
2. **WHERE**: throw away rows that do not match.
3. **GROUP BY**: collapse the remaining rows into groups.
4. **HAVING**: throw away groups that do not match.
5. **SELECT**: compute the output columns.

6. **ORDER BY**: sort the result.
7. **LIMIT / OFFSET**: cut the result down.

This explains things that otherwise feel like arbitrary rules. You cannot use a column alias from SELECT inside WHERE, because WHERE runs before SELECT, so the alias does not exist yet. You can use that alias in ORDER BY, because ORDER BY runs after SELECT. WHERE filters rows before grouping, so you cannot put an aggregate like `count(*)` in it; HAVING filters groups after grouping, which is why aggregates belong there. When a query gives an error that feels unfair, check this list first.

Note that "logically" matters. The planner is free to reorder work, push filters down, and skip steps, as long as the result is the same as if it ran in this order. You will study how it really does it next week with EXPLAIN. This week you only need the logical order.

SELECT and expressions

SELECT is not only for picking columns. It computes things: `price * quantity`, `lower(email)`, `now() - created_at`. Anything that produces one value per row can go there. `SELECT *` returns every column, which is fine while exploring and a bad habit in application code, because adding a column later silently changes what your query returns.

`DISTINCT` removes duplicate rows from the output. It is often a sign of a problem: people add it to hide duplicates caused by a join they did not understand. If you reach for DISTINCT to fix a result, stop and ask where the duplicates came from.

WHERE: filtering rows

WHERE takes a condition that is true, false, or unknown, and keeps only rows where it is true. The common tools are comparison (`=`, `<>`, `<`, `>=`), `BETWEEN` (inclusive on both ends), `IN (a, b, c)`, `LIKE` and `ILIKE` for patterns (`%` means any characters, `_` means one character, `ILIKE` ignores case), and `AND`, `OR`, `NOT` to combine them. Use parentheses whenever you mix AND and OR, because AND binds tighter and the result is often not what you read.

NULL and three-valued logic

This is the part of SQL that surprises experienced engineers the most. NULL does not mean zero or empty. It means "unknown" or "no value." Any comparison with NULL gives neither true nor false but unknown. So `NULL = NULL` is unknown, not true. `5 > NULL` is unknown. `NOT unknown` is still unknown.

WHERE keeps only rows that are true. Unknown is dropped, just like false. That has real consequences. In your customers table, `marketing_opt_in` is NULL for about 40% of rows. If you write `WHERE marketing_opt_in <> true` to find people who did not opt in, you get only the rows that are explicitly false. The NULL rows vanish, because `NULL <> true` is unknown. Nothing warns you. The query runs and returns a smaller number than you expected.

The tools for dealing with it: `IS NULL` and `IS NOT NULL` to test for it, `IS DISTINCT FROM` as a comparison that treats NULL as a normal value (`NULL IS DISTINCT FROM true` is true), and `COALESCE(x, default)` to replace NULL with a fallback. A habit worth building early: every time you write a filter on a nullable column, ask what happens to the NULL rows.

`NOT IN` has a nasty version of this. If the list you are checking against contains even one NULL, `x NOT IN (...)` can never be true, and you get zero rows back. `NOT EXISTS` does not have this problem, which is why it is usually the safer way to write "rows with no match."

ORDER BY, LIMIT and OFFSET

Without ORDER BY, Postgres guarantees no row order at all. It may look consistent, and then change after an update or a vacuum. If order matters, say so. ORDER BY accepts multiple columns, `DESC`, and `NULLS FIRST` or `NULLS LAST`. By default Postgres sorts NULLs as if they were larger than everything, so they come last in ascending order and first in descending order.

LIMIT cuts the result to N rows. It is only meaningful together with ORDER BY, otherwise you get an arbitrary N. OFFSET skips rows, and is the usual way to do pagination (`LIMIT 20 OFFSET 40` for page 3). The problem is that OFFSET still has to produce and throw away all the skipped rows, so deep pages get slower and slower, and rows can shift between pages if data changes. The alternative is keyset pagination: remember the last row you showed and ask for "rows after this one" (`WHERE id > 12345 ORDER BY id LIMIT 20`). It stays fast on any page, and it is what production systems use for large tables.

Changing data: INSERT, UPDATE, DELETE

`INSERT INTO table (col1, col2) VALUES (...)` adds rows. You can insert many rows in one statement, or insert the result of a query with `INSERT ... SELECT`. Always list the column names; relying on column order breaks the day someone adds a column.

`UPDATE table SET col = value WHERE condition` changes existing rows. `DELETE FROM table WHERE condition` removes them. The classic disaster is forgetting the WHERE clause, which changes or deletes every row. The habit that protects you: write the WHERE first, run it as a SELECT to see exactly which rows it matches, and only then turn it into an UPDATE or DELETE. While learning, wrap changes in a transaction (`BEGIN; ... ROLLBACK;`) so you can look at the effect and undo it. Transactions get their full treatment next week.

All three can end with `RETURNING`, which gives back the affected rows. `INSERT ... RETURNING id` is how you get the generated id of a new row without a second query. This is a Postgres feature, and one you will use constantly.

UPSERT

UPSERT means "insert this row, or if it already exists, update it instead." In Postgres it is `INSERT ... ON CONFLICT`:

```
INSERT INTO products (sku, name, price, status, created_at)
VALUES ('SKU-000001', 'Widget', 9.99, 'active', now())
ON CONFLICT (sku)
DO UPDATE SET price = EXCLUDED.price;
```

`ON CONFLICT (sku)` names the unique column or constraint that defines "already exists." `EXCLUDED` is a special name for the row you tried to insert, so you can use its values in the update. The alternative is `DO NOTHING`, which silently skips the row if it exists. UPSERT is atomic: two sessions doing it at the same time will not create duplicates or crash with a race, which is why it beats the "check if it exists, then insert" pattern you might write in application code. That pattern has a gap between the check and the insert where another session can slip in.

2. Joins

A relational database splits data across tables so each fact is stored once. A customer lives in `customers`, an order lives in `orders`, and the order points at the customer through `customer_id`. A join is how you put them back together in a query. Almost every real query on a real schema contains at least one.

What a join actually does

Conceptually, a join takes two sets of rows and pairs them up according to a condition. The condition goes in `ON`:

```
SELECT c.full_name, o.id, o.total
FROM customers c
JOIN orders o ON o.customer_id = c.id;
```

Think of it as: for every customer row, find the order rows whose `customer_id` equals the customer's `id`, and output one combined row for each pair. If a customer has 3 orders, that customer appears 3 times in the output. That sentence is the root of the most common join bug, so remember it: **a join produces one output row per matching pair, not one per row on either side.**

Table aliases (`c`, `o`) are not optional decoration. When two tables share a column name like `id`, you must say which one you mean, and short aliases keep the query readable.

The join types

INNER JOIN (just `JOIN` means inner) keeps only pairs that match. A customer with no orders disappears from the result. An order whose customer does not exist also disappears.

LEFT JOIN keeps every row from the left table, and fills the right side's columns with NULL when there is no match. Use it when you want "all customers, and their orders if they have any." Customers with no orders appear once, with NULLs in the order columns.

RIGHT JOIN is the mirror image: keep every row from the right table. It is rarely used, because you can always swap the table order and write a LEFT JOIN instead, which reads more naturally.

FULL JOIN keeps every row from both sides. Matched rows are combined, and unmatched rows from either side appear with NULLs on the other side. It is useful for reconciliation: comparing two lists and seeing what is only in one or only in the other.

CROSS JOIN pairs every row on the left with every row on the right, with no condition. 100 rows times 50 rows gives 5,000 rows. It sounds dangerous, and on big tables it is, but it is the right tool for building combinations, like every day of a month crossed with every product, which you then LEFT JOIN real data onto so that empty days still show up.

Self-join is not a keyword, just a technique: you join a table to itself using two different aliases. It is how you handle hierarchies stored in one table. In `employees`, each row has a `manager_id` that points at another row in the same table, so `FROM employees e JOIN employees m ON e.manager_id = m.id` gives each employee next to their manager. The same works for `customers.referred_by`.

ON versus WHERE, especially with LEFT JOIN

With an INNER JOIN, a condition in `ON` and a condition in `WHERE` give the same result, so people mix them freely. With a LEFT JOIN they are not the same, and this is a classic trap.

A condition in `ON` decides which rows *match*. The left table's rows are kept regardless. A condition in `WHERE` runs after the join and filters the combined rows. If you LEFT JOIN customers to orders and then write `WHERE o.status = 'paid'`, the customers with no orders have NULL in `o.status`, the comparison is unknown, and those customers are removed. You have turned your LEFT JOIN back into an INNER JOIN without noticing. If you want "all customers, with only their paid orders attached," the condition belongs in the ON clause: `LEFT JOIN orders o ON o.customer_id = c.id AND o.status = 'paid'`.

NULL in joins

A join condition `a.x = b.y` is never true when either side is NULL, because NULL equals nothing, not even NULL. So rows with a NULL in the join column never match, and in an inner join they silently disappear. In your data, `products.category_id` is NULL for about 3% of products. Join products to categories with an inner join and those products are gone. A LEFT JOIN keeps them. When you want NULLs to match each other, use `IS NOT DISTINCT FROM` as the join condition, though you will need it rarely.

Fan-out: the bug that gives wrong totals

Fan-out is what happens when you join a table to another table that has many rows per key, and then aggregate something from the first table. Take an order with 3 items and 2 payment rows. Join orders to items and to payments, and that single order becomes 3 × 2 = 6 rows. Now `sum(o.total)` counts that order's total six times. The query runs, the numbers look plausible, and the revenue figure is wrong.

The defence is to know the cardinality of every join before you write it. Is it one-to-one, one-to-many, many-to-many? When you join two different "many" sides to the same parent, they multiply. The fix is to aggregate each many-side separately, in a subquery or

CTE, down to one row per parent key, and then join those results. Your seed data has payments with 0 to 3 rows per order, deliberately, so you can see this happen.

A good habit: after writing a join, compare the row count to what you expect. If you joined orders to items and expected roughly one row per item, check that. Surprise in the count is the warning sign.

Anti-joins and semi-joins

A question like "which customers have never ordered?" is an anti-join. There are three ways to write it. `LEFT JOIN ... WHERE o.id IS NULL` joins and then keeps the rows where no match was found. `NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id)` asks directly. `NOT IN (SELECT customer_id FROM orders)` also works, until the subquery returns a NULL, at which point it returns nothing. Prefer `NOT EXISTS` or the LEFT JOIN form.

The opposite, "customers who have at least one order," is a semi-join, written with `EXISTS`. It is better than joining and then using DISTINCT, because `EXISTS` stops at the first match and never multiplies rows.

Join order

For INNER JOINs, the order you write the tables in does not change the result, and Postgres is free to reorder them for speed. For outer joins (LEFT, RIGHT, FULL), order matters, because "left" and "right" mean something and the semantics depend on it. You do not need to optimise join order this week. Just be aware that when you chain several joins, each one is applied to the result of the previous ones, so a LEFT JOIN followed by an INNER JOIN on the same right-side table can cancel the LEFT JOIN, the same way the WHERE trap does.

3. Aggregation

Aggregation means collapsing many rows into one summary value: a count, a sum, an average, a maximum. It is how you answer questions about groups of things rather than individual rows: how many orders per customer, revenue per month, the most expensive product per category.

Aggregate functions

The core ones are `count`, `sum`, `avg`, `min` and `max`. Postgres also has `string_agg` (join text values with a separator), `array_agg` (collect values into an array), `bool_and` and `bool_or`, and statistical ones like `percentile_cont` for medians and percentiles.

Without a GROUP BY, an aggregate collapses the whole table into a single row: `SELECT count(*), sum(total) FROM orders` returns one line. With GROUP BY, you get one row per distinct group value:

```
SELECT status, count(*) AS orders, sum(total) AS revenue
FROM orders
GROUP BY status;
```

The rule that trips people up: every column in SELECT must either appear in GROUP BY or be inside an aggregate function. If you select `customer_id` and `total` and group by `customer_id`, Postgres asks what to do with all the different totals in a group, and refuses until you say `sum(total)` or `max(total)` or similar. This is not pedantry. The database genuinely does not know which of the many values you want.

How aggregates treat NULL

This is the second half of NULL semantics and it matters just as much. Aggregate functions ignore NULLs. `avg(discount_pct)` averages only the rows where a discount exists, so if 85% of order lines have a NULL discount, the average is over the other 15%. That might be what you want or it might be badly misleading. If you want NULL treated as zero, say so: `avg(coalesce(discount_pct, 0))`.

`count(*)` counts rows. `count(column)` counts only rows where that column is not NULL. They give different numbers on a nullable column, and people get this wrong constantly. `count(DISTINCT column)` counts distinct non-NULL values.

An aggregate over zero rows is NULL for `sum`, `avg`, `min` and `max`, but `count` returns 0. So `sum(total)` over an empty result gives NULL, not 0, which can break application code that expects a number. Wrap it as `coalesce(sum(total), 0)` when that matters.

Also, in GROUP BY, all NULLs in the grouping column land together in one group. A NULL group shows up as its own row. In `customers`, grouping by `country_code` gives you a group for NULL, even though NULL never equals NULL anywhere else. GROUP BY treats NULLs as the same, which is a deliberate exception.

WHERE versus HAVING

WHERE filters rows before they are grouped. HAVING filters groups after the aggregates are computed. "Customers who placed more than 10 orders" needs HAVING, because the count only exists after grouping:

```
SELECT customer_id, count(*) AS n
FROM orders
GROUP BY customer_id
HAVING count(*) > 10;
```

If a condition does not involve an aggregate, put it in WHERE, not HAVING. It is clearer, and it lets the database throw rows away before it does the grouping work.

FILTER: conditional aggregates

Often you want several different counts from the same pass over the data: total orders, paid orders and refunded orders per customer. Instead of three queries, use `FILTER`:

```
SELECT customer_id,
       count(*) AS all_orders,
       count(*) FILTER (WHERE status = 'refunded') AS refunded
FROM orders
GROUP BY customer_id;
```

Each aggregate gets its own filter. It is the clean replacement for the older `sum(CASE WHEN ... THEN 1 ELSE 0 END)` trick, and it works on any aggregate, not only count.

Grouping sets, ROLLUP and CUBE

Sometimes you want the same measure at several levels of detail in one result: revenue per country and city, plus a subtotal per country, plus a grand total. You could write three queries and UNION them. `GROUPING SETS` does it in one.

`GROUPING SETS ((a, b), (a), ())` means "group by a and b, and also by a alone, and also by nothing at all (the grand total)." `ROLLUP (a, b)` is a shortcut for exactly that hierarchy: (a, b), (a), (). `CUBE (a, b)` gives every combination: (a, b), (a), (b), ().

In the subtotal rows, the columns that were rolled up show as NULL. That creates an ambiguity: is that NULL a subtotal marker, or a real NULL in the data (like a customer with no country)? The `GROUPING(column)` function settles it. It returns 1 when the column was

rolled up in that row and 0 when it is a real group value. Use it whenever you use ROLLUP or CUBE on a nullable column.

DISTINCT ON

`DISTINCT ON` is a Postgres-specific feature for the very common "latest row per group" problem. For example, each customer's most recent order:

```
SELECT DISTINCT ON (customer_id) customer_id, id, placed_at
FROM orders
ORDER BY customer_id, placed_at DESC;
```

It keeps the first row for each distinct `customer_id`, where "first" is decided by the ORDER BY. The ORDER BY must start with the same columns as the DISTINCT ON, and the columns after that decide which row wins. It is short and fast to write. Its limits: it is not standard SQL, so it will not work on other databases, and it can only return one row per group. For "top 3 per group" you need window functions or LATERAL, covered in the next section.

Aggregating after a join

This connects back to fan-out. If you aggregate over a join that multiplied your rows, the aggregate is wrong. The reliable pattern is to aggregate each table down to the level you need first (one row per order, say), and join those summaries afterwards. Getting into the habit of asking "what is one row of this intermediate result?" before aggregating will save you from most reporting bugs.

4. Advanced queries

This section is where SQL stops being a lookup tool and becomes a real programming language. Everything here builds on the earlier sections, so if joins or NULL handling still feel shaky, go back before reading on.

Subqueries

A subquery is a query inside another query, in parentheses. It comes in three useful flavours.

A **scalar subquery** returns exactly one value and can be used anywhere a single value fits: `WHERE total > (SELECT avg(total) FROM orders)`. If it returns more than one row, Postgres errors.

A subquery used with **IN** returns a list: `WHERE customer_id IN (SELECT id FROM customers WHERE country_code = 'DE')`.

A subquery in **FROM** acts like a temporary table: `FROM (SELECT customer_id, count(*) AS n FROM orders GROUP BY customer_id) t`. It must have an alias. This is the standard way to aggregate something before joining it, which is the cure for fan-out.

Correlated subqueries and EXISTS

A normal subquery runs independently of the outer query. A **correlated** subquery refers to a column of the outer query, so conceptually it is re-evaluated for each outer row:

```
SELECT c.id
FROM customers c
WHERE EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.id AND o.status = 'refunded'
);
```

For each customer, ask: is there at least one refunded order? `EXISTS` returns true as soon as the subquery finds one row, so what you put in its SELECT does not matter (`SELECT 1` is the convention). Postgres is usually smart enough to turn this into a proper join internally, so do not assume a correlated subquery is slow, but a correlated subquery in the SELECT list that does real work per row on a large table is worth being suspicious of.

A rule of thumb for choosing: use `EXISTS` for "has at least one," `NOT EXISTS` for "has none," `IN` for short lists of values, and a join when you actually need columns from the other table.

CTEs (WITH queries)

A Common Table Expression gives a name to a subquery so you can read the query top to bottom instead of inside out:

```
WITH order_totals AS (
  SELECT order_id, sum(quantity * unit_price) AS items_total
  FROM order_items
  GROUP BY order_id
)
SELECT o.id, o.total, t.items_total
FROM orders o
JOIN order_totals t ON t.order_id = o.id
WHERE o.total <> t.items_total;
```

You can chain several CTEs, each able to use the ones before it. Their main value is readability: you break a complicated query into named steps. Before Postgres 12, every CTE was computed once and stored, acting as a barrier the planner could not see through. From 12 on, a CTE used once is treated like an inlined subquery, and only materialised if it is referenced more than once or you write `WITH x AS MATERIALIZED (...)`. You can force inlining with `NOT MATERIALIZED`. The practical takeaway: a CTE is mostly a readability tool, not a performance tool, and you should be able to say which of the two you are using it as.

CTEs can also contain INSERT, UPDATE or DELETE with `RETURNING`, so one statement can, say, delete rows and insert them into an archive table in a single step. These are called data-modifying CTEs. Related: `UPDATE ... FROM` and `DELETE ... USING` let you update or delete one table based on a join to another.

Recursive CTEs

A recursive CTE lets a query refer to its own output, which is how you walk hierarchies of unknown depth: an org chart, a category tree, a chain of referrals. Plain joins cannot do this, because they need to know in advance how many levels there are.

The structure is always two parts joined by `UNION ALL`:

```
WITH RECURSIVE chain AS (
  -- 1. anchor: the starting rows
  SELECT id, manager_id, 1 AS depth
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- 2. recursive step: find the next level, using what we have so far
  SELECT e.id, e.manager_id, c.depth + 1
  FROM employees e
  JOIN chain c ON e.manager_id = c.id
)
SELECT * FROM chain;
```

Postgres runs the anchor once, then repeatedly runs the recursive part against the rows produced by the previous round, until a round produces no new rows. Each round is one level deeper. You can carry extra columns along, like `depth`, or a path built up as an array or string, which is how you print "CEO > VP > Manager > You."

The danger is a cycle. If row A leads to B, B leads to C and C leads back to A, the recursion never ends. Your seed data has a deliberate referral cycle among customers 99991, 99992 and 99993 for this reason. Two ways to guard against it: keep an array of the ids you have already visited and refuse to step onto one of them, or use the `CYCLE` clause that Postgres 14 added, which does the tracking for you. In the real world, always think about termination before running a recursive query on data you do not control.

Window functions

Window functions are the biggest single jump in SQL power. An aggregate with GROUP BY collapses rows into one per group. A window function computes the same kind of thing but **keeps every row**, attaching the result as an extra column. That is the whole idea.

```
SELECT id, customer_id, total,
       sum(total) OVER (PARTITION BY customer_id) AS customer_total
FROM orders;
```

Every order row is still there, and each one also shows the total for its customer. The `OVER (...)` clause defines the window, and it has three parts you can combine:

- `PARTITION BY` splits the rows into independent groups, like GROUP BY but without collapsing. Leave it out and the window is the entire result.
- `ORDER BY` inside OVER sorts rows within each partition. For aggregates it also turns the calculation into a running one.
- A **frame** says exactly which rows around the current row are included, like "the previous 6 rows plus this one."

There are three families of window functions. **Ranking functions**: `row_number()` gives 1, 2, 3 with no ties; `rank()` gives ties the same number and then skips (1, 1, 3); `dense_rank()` ties without skipping (1, 1, 2). **Offset functions**: `lag(x)` and `lead(x)` read the value from the previous or next row, which is how you compute "change since the last order" or "time between events." **Aggregates as windows**: `sum`, `avg`, `count` and others used with OVER, giving running totals and moving averages.

Ordering inside the window with an aggregate gives a running total: `sum(total) OVER (PARTITION BY customer_id ORDER BY placed_at)`. This works because, with an ORDER BY, the default frame is "from the start of the partition up to the current row." That default frame is also a trap. With `last_value(x)`, the default frame stops at the current row, so you get the current row's value, not the last in the partition. You must widen the frame explicitly: `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

Frames come in three modes. `ROWS` counts physical rows. `RANGE` groups rows with equal ORDER BY values together, so ties share a frame, which can surprise you when timestamps collide. `GROUPS` counts groups of equal values. For most work, `ROWS` is the one you want, and the one that behaves predictably.

When does it run? Logically after WHERE, GROUP BY and HAVING, and before the final ORDER BY and LIMIT. That is why you cannot filter on a window function result directly in WHERE. To get "the top order per customer," compute `row_number()` in a subquery or CTE, then filter `WHERE rn = 1` outside.

A classic use is **gaps and islands**: grouping consecutive events together, such as splitting a user's events into sessions whenever there is more than a 30-minute gap. The method is to use `lag()` to find where a gap starts, turn that into a 0/1 flag, and take a running `sum` of the flag to give each session its own number.

LATERAL

Normally, a subquery in FROM cannot see the other tables in the same FROM clause. `LATERAL` lifts that rule: the subquery can refer to columns from the tables to its left, and it is re-run for each of their rows. Think of it as a `for each` loop inside SQL.

```
SELECT c.id, o.id AS order_id, o.total
FROM customers c
CROSS JOIN LATERAL (
  SELECT id, total
  FROM orders
  WHERE customer_id = c.id
  ORDER BY total DESC
  LIMIT 3
) o;
```

For each customer, run the inner query and get that customer's top three orders. That inner `LIMIT 3` applies per customer, which an ordinary join cannot do. Use `LEFT JOIN LATERAL (...) ON true` if you want customers with no orders to still appear.

Top-N per group is the textbook case, and it can also be done with `row_number()` in a window. The two approaches give the same result, and each has a place: window functions read well and are good when you need most of the data anyway, while LATERAL is strong when you want only a few rows per group from a large table. LATERAL is also how you expand arrays or JSON arrays per row, with functions like `unnest()` and `jsonb_array_elements()`, since those functions need to see the row they belong to.

5. Data types

Every column has a type, and the type decides what values are allowed, how much space they take, how they compare and sort, and which functions work on them. Choosing a type is a design decision with consequences, because changing it later on a big table is slow and risky. Postgres is strict about types, which is a feature: it rejects bad data at the door.

Numbers

Integers come in three sizes: `smallint` (2 bytes, up to about 32 thousand), `integer` (4 bytes, up to about 2.1 billion) and `bigint` (8 bytes, up to about 9.2 quintillion). Use `integer` for ordinary counts and ids, and `bigint` for anything that could grow past 2

billion, like an events table or an order id on a busy system. Running out of an integer id on a live table is a painful outage, so when in doubt for a primary key on a growing table, pick `bigint`.

numeric(p, s) stores exact decimal numbers. `p` is the total number of digits and `s` is how many come after the decimal point, so `numeric(12,2)` holds up to 10 digits before the point and 2 after. It is exact but slower than floats. If you insert a value with too many decimals, Postgres rounds it silently. If you insert one with too many digits before the point, it raises an error. Use `numeric` for money and anything where being off by a cent matters.

real and double precision are binary floating point. They are fast and compact but approximate: `0.1 + 0.2` does not equal `0.3` exactly, and sums over millions of values drift. Use them for measurements, scores and scientific data, never for money and never in a column you compare with `=`.

One trap with integers: dividing two integers gives an integer, so `5 / 2` is `2`. Cast one side (`5 / 2.0` or `5::numeric / 2`) when you want a fraction.

Text

`text` stores strings of any length. `varchar(n)` stores strings up to n characters and rejects longer ones. In Postgres they perform identically, so the old advice to prefer `varchar(n)` for speed does not apply. Use `text`, and add a `CHECK` constraint if you actually need a length limit. `char(n)` pads with spaces and is almost never what you want.

Comparisons are case-sensitive, so `'Bob'` and `'bob'` are different values. For case-insensitive matching use `ILIKE`, compare `lower(a) = lower(b)`, or use the `citext` extension. Collation, meaning the rules for sorting and comparing text in a given language, also affects results. Your customers table has emails that differ only by case or surrounding whitespace, which is a deliberate example of what happens when nothing enforces a clean format.

Boolean

`boolean` holds true, false or NULL. Remember that a nullable boolean has three states, and that `WHERE flag <> true` excludes NULLs. If you do not want a third state, add `NOT NULL`.

Dates, times and time zones

This is the area where experienced engineers most often get things wrong, so it deserves care.

Postgres has `date` (a calendar day), `time`, `timestamp` (also written `timestamp without time zone`) and `timestamptz` (`timestamp with time zone`), plus `interval` for durations.

The names are misleading. **`timestamptz` does not store a time zone.** It stores a single instant on the universal timeline, internally as UTC. When you insert a value, Postgres converts it from whatever zone is in the text, or from your session's time zone setting, into UTC. When you read it, Postgres converts it back into your session's time zone for display. So the same stored instant shows as 11:32 in Tokyo and 22:32 the night before in New York. It is one moment shown on different clocks.

`timestamp` (without time zone) stores only a clock reading, like "2026-08-14 02:32:57," with no information about which zone that clock was in. It is not converted on input or output. If you store UTC in it, nothing in the database says so, and the next person has to guess.

The practical rule that most experienced teams follow: **use `timestamptz` for any real moment in time** (when an order was placed, when a login happened), and use `timestamp` or `date` only for things that are genuinely wall-clock, like "the store opens at 9:00 local time every day."

The operator `AT TIME ZONE` converts, and it works in opposite directions depending on its input. On a `timestamptz`, `x AT TIME ZONE 'Asia/Tokyo'` gives you a plain `timestamp` showing the Tokyo wall-clock time for that instant. On a plain `timestamp`, it says "this clock reading was in that zone" and gives you a `timestamptz`. The zone you name always describes where the value *came from or should be shown*, so it has to match the truth about your data, not the viewer's location.

Comparing a `timestamp` with a `timestamptz` makes Postgres silently convert the plain one using the session's time zone. That means the same query can return different answers for different sessions, which is the exact trap in your `orders.shipped_at` column. Always write the `AT TIME ZONE` yourself so the meaning is in the query and not in a setting.

Two more rules. Use named zones like `'America/New_York'`, never fixed offsets like `-05:00`, because named zones carry daylight saving rules and offsets do not. And remember that days are not always 24 hours: on the US spring-forward day a day has 23

hours, and on the fall-back day it has 25. Grouping by day means choosing whose day, and a customer's local day is not the UTC day. `date_trunc('day', ts AT TIME ZONE 'America/New_York')` gives you New York days.

JSON and JSONB

Postgres has two JSON types. `json` stores the exact text you gave it. `jsonb` stores a parsed binary form: it ignores whitespace, drops duplicate keys, does not preserve key order, and can be indexed and searched efficiently. Use `jsonb` unless you have a specific reason to keep the original text.

The operators you need: `->` gets a field and returns JSON; `->>` gets a field and returns text; `#>` and `#>>` follow a path through nested objects; `@>` tests whether the left value contains the right one (`profile @> '{"plan": "pro"}'`); `?` tests whether a key exists. For arrays inside JSON, `jsonb_array_elements` turns them into rows. Functions like `jsonb_typeof` tell you whether a value is an object, array, string, number, boolean or null.

Watch the two kinds of null. A key missing from the object, a key present with the JSON value `null`, and a SQL NULL in the column are three different things. `payload->>'ref'` returns SQL NULL for both a missing key and a JSON null, so you cannot tell them apart with `->>` alone. Use `?` to test for the key and `jsonb_typeof` to check for a JSON null. Also note that `->>` always returns text, so numbers inside JSON must be cast: `(payload->>'amount')::numeric`. If the same key holds a number in some rows and a string in others, as in your events data, a cast can fail or silently mislead.

JSONB versus real columns is the design question that matters. JSONB is good for data whose shape varies or is not known in advance, like product attributes that differ by category, or raw payloads from an outside service you want to keep as received. It is poor for data you filter, join, sum or enforce rules on regularly. A JSONB field cannot have a foreign key, a NOT NULL on an inner field, a type check on an inner field, or good statistics for the planner. The usual healthy design is real columns for the fields you query and constrain, and a JSONB column for the leftover flexible part. When a key keeps appearing in WHERE clauses, it should probably become a column.

Arrays

Postgres lets a column hold an array of any type, like `text[]` for tags. The useful operators are `= ANY(arr)` (is this value in the array), `@>` (does the array contain these values), `&&` (do two arrays overlap), and `unnest(arr)` (expand an array into rows). Arrays are convenient for small, simple lists that belong to one row and are never queried on their

own. They are the wrong choice when the items have their own attributes or need to be joined to other tables; that is what a separate table is for. Note that `NULL` and an empty array `'{}'` are different, and a NULL array does not match `@>` or `ANY`, so rows with NULL tags are silently skipped by those filters.

Enums

An enum is a type with a fixed list of allowed values: `CREATE TYPE order_status AS ENUM ('pending', 'paid', 'shipped')`. It stores compactly, rejects invalid values, and sorts in the order you declared. The cost is flexibility. You can add values easily, but you cannot remove or reorder them without a painful migration. An alternative is a `text` column with a `CHECK` constraint, or a small lookup table with a foreign key. Enums fit values that are truly stable; lookup tables fit values the business might change.

UUIDs

`uuid` is a 128-bit identifier, usually written as 36 characters like `3f2a9c1e-5b7d-4e8a-9c21-7d4e5f6a8b90`. Its appeal is that you can generate one anywhere, in the app or in another system, without asking the database, and collisions are practically impossible. `gen_random_uuid()` makes a random one. The costs are size (16 bytes against 4 or 8 for integers) and, with random UUIDs as primary keys, poor insert locality in the index, which matters at large scale. Time-ordered UUID versions reduce that problem. Use UUIDs when ids must be created outside the database or must not be guessable, and plain integers when neither applies.

Casting

`value::type` or `CAST(value AS type)` converts between types. Postgres converts some types implicitly, but relying on that is where subtle bugs come from, as with the timestamp comparison above. Be explicit about conversions, and check `pg_typeof(expression)` whenever you are unsure what type something really is.

6. Schema design

A schema is the set of tables, columns, types and rules that describe your data. Good schema design is mostly about one goal: make it impossible to store wrong data, and make each fact live in exactly one place. A query can be rewritten tomorrow, but bad data already in the table is very hard to fix, and a bad schema shapes every query written against it for years.

Primary keys

A primary key uniquely identifies each row. It is automatically `NOT NULL` and `UNIQUE`, and it gets an index. Every table should have one. The choice is between a **natural key**, a value that already exists in the real world (an email, an ISBN, a country code), and a **surrogate key**, an id with no meaning that the database generates. Natural keys sound appealing but real-world values change, get reused or turn out not to be unique: people change emails, and your own data has the same email in different cases. Most application tables use a surrogate key (`bigint GENERATED ALWAYS AS IDENTITY` or a UUID) and enforce the natural identifier separately with a `UNIQUE` constraint.

A **composite primary key** uses two or more columns together. `order_items` uses `(order_id, line_no)`: neither alone is unique, but the pair is. This is common for tables that belong to a parent.

Prefer `GENERATED ... AS IDENTITY` over the older `serial` type. It does the same job but is standard SQL and avoids some permission and ownership oddities.

Foreign keys

A foreign key says "this column must match an existing row in that other table." `orders.customer_id REFERENCES customers(id)` means you cannot create an order for a customer that does not exist, and you cannot delete a customer who still has orders (unless you say otherwise). It is the database refusing to store a broken reference, and it is the main reason to use a relational database at all. Without it, the application is the only thing stopping orphan rows, and applications have bugs and are not the only thing that writes to a database.

A point that catches people: Postgres automatically indexes a primary key, but it does **not** automatically index a foreign key column. That has consequences for joins and deletes on the child table, and next week's material on indexes explains why. This week, just remember that the foreign key columns in the shop schema are deliberately left without indexes.

Cascades

What should happen to child rows when a parent row is deleted or its key changes? The `ON DELETE` option decides:

- `NO ACTION` or `RESTRICT` (the default): refuse the delete if children exist. `RESTRICT` checks immediately; `NO ACTION` can be deferred to the end of the transaction.

- `CASCADE`: delete the children too. Right for things that have no meaning without the parent, like order lines when an order is deleted. Dangerous when used loosely, because one delete can quietly remove thousands of rows through a chain of cascades.
- `SET NULL`: keep the children and blank out the reference. Needs a nullable column.
- `SET DEFAULT`: set the reference to the column's default value.

Choose deliberately for each relationship by asking what the child means without its parent. Note that in your shop schema, `order_items` cascades from `orders`, but `orders` does not cascade from `customers`, so deleting a customer who has orders is blocked. That is also why soft deletes (a `deleted_at` column) are common: the row stays, history stays intact, and you filter it out in queries.

Constraints

Constraints are rules the database enforces on every write, whatever application or script does the writing.

- `NOT NULL`: the column must have a value.
- `UNIQUE`: no two rows can share the value. A unique constraint allows multiple NULLs by default, because NULLs are never equal to each other.
- `CHECK`: any condition on the row, like `price >= 0` or `discount_pct BETWEEN 0 AND 100`. Powerful and underused.
- `DEFAULT`: a value used when none is supplied.
- `EXCLUDE`: a more general form of unique, useful for things like "no two bookings for the same room can overlap in time."

A good habit is to put as many rules as possible in the schema. Every rule left to application code is a rule that some other code path will eventually break. You will see the opposite in your seed data: `customers.email` has no unique constraint, and the result is the duplicates you can find.

Nullability

Every column should be `NOT NULL` unless you can state a real reason it may be missing. NULL is not a neutral choice, because everything you learned earlier about three-valued logic, joins and aggregates applies to every nullable column forever. Ask what NULL would mean: "we do not know," "it does not apply," or "not yet." If it can mean several of those at

once, the column is ambiguous and probably needs a better design, such as a separate status column or a separate table. For example, `shipped_at` being NULL could mean "not shipped yet" or "shipped but we lost the timestamp," and you cannot tell which.

Normalization

Normalization is the practice of organising tables so that each fact is stored once. The reason is **anomalies**, the problems that appear when a fact is stored in several places. If a customer's address is copied onto every order row, then changing it means updating thousands of rows (update anomaly), you cannot store a customer who has not ordered yet (insert anomaly), and deleting the last order loses the address (delete anomaly).

The normal forms are a sequence of rules for avoiding these. In plain terms:

- **First normal form (1NF):** each column holds a single value, and there are no repeating groups. A comma-separated list of tags in one text column breaks this; a separate row per tag, or a proper array used with care, does not.
- **Second normal form (2NF):** every non-key column depends on the whole key, not just part of a composite key. If `order_items` held the product name, it would depend only on `product_id`, not on `(order_id, line_no)`, so the name belongs in `products`.
- **Third normal form (3NF):** non-key columns depend on the key and nothing else. If `customers` stored both `country_code` and `country_name`, the name depends on the code, not on the customer, so it belongs in a `countries` table.

A useful summary people repeat: every column should depend on the key, the whole key, and nothing but the key. For most application schemas, 3NF is the right target. You will rarely need the higher forms.

A few relationship shapes to recognise. **One-to-many** is a foreign key on the "many" side (one customer, many orders). **Many-to-many** needs a third "junction" table holding two foreign keys (products and categories, if a product could belong to several categories). **One-to-one** is a foreign key with a `UNIQUE` constraint, used to split off rarely needed or sensitive columns.

When to denormalize

Denormalizing means deliberately storing a fact in more than one place, trading the guarantees of normalization for speed or simplicity. Your schema has an example: `orders.total` is the sum of that order's items, stored on the order so you do not have to

recompute it for every read. That makes common queries cheaper, and it also creates the risk you can see in your data: about 1% of totals no longer match their items, because something updated one without the other.

The rules for doing it responsibly: normalize first, and denormalize only when you have measured a real problem. Know where the single source of truth is. Decide how the copy stays correct, such as a trigger, a transaction in the application, or a periodic reconciliation job, and write that down. And be able to detect drift, which is exactly the reconciliation query you will write this week. Common legitimate cases include stored totals and counters, snapshotting a price on an order line at the time of purchase (`order_items.unit_price` is that, and it is correct to copy, because the product's price will change later while the historical order must not), and reporting tables built for read speed.

Note that last example, because it shows the real test. A copied value is a bug when it is supposed to track the source, and a feature when it is supposed to record a moment in time.

Putting it together

When you look at a schema, run through this list: does every table have a primary key? Is every relationship backed by a foreign key? What happens to children on delete? Which columns are nullable, and is each one justified? Are there CHECK or UNIQUE constraints for the rules the business actually has? Are types right for the data (timestamptz for moments, numeric for money, an enum or lookup for fixed sets)? Where is data copied, and what keeps the copies honest? You will do exactly this audit on the shop schema as part of the practice.